

## SELECTION SORT



Read the way code of sorting  
→ comparing the array elements on changing index.  
→ swap the array elements on changing index.

\* Bubble pushes the largest element to the back.  
Selection pulls the smallest element to the front.

Selection Sort is a simple comparison-based sorting algorithm.

Why is it called "Selection" Sort?  
Because in every pass we select the smallest element from the remaining unsorted part -> generally like Bubble Sort.  
We don't swap repeatedly like Bubble Sort.  
We first select, then swap once.

### STEPS TO FOLLOW:

For array of size n:

- Start from index 0.
- Find smallest element from index 0 to n-1.
- Swap it with index 0.
- Repeat until i = n-1.

We don't need to check last element, it's automatically sorted.

Number:  
We'll keep pass pass.  
After each pass, left side becomes sorted.

- Outer loop finds the position.
- Inner loop finds the smallest.
- Swap, if smaller element found.

Three major pointers used:  
1. minIndex: position where smallest should go.  
2. min: value of smallest element.  
3. i: current position.

No need to remember full code, to have it in your mind:

```
for i = 0 to n-2  
  minIndex = i  
  for j = i+1 to n-1  
    if arr[j] < arr[minIndex]  
      swap(i, minIndex)
```

Pseudocode

Time Complexity:  
Best case:  $O(n^2)$   
Average case:  $O(n^2)$   
Worst case:  $O(n^2)$   
Unlike Bubble Sort, it does NOT improve in best case.

Space Complexity:  $O(1)$   
No extra array used, just few temporary variables.

Selection sort is "in-place" but it is NOT "stable" by default.

The sorting process is performed using only a few extra variables to keep track of values and indices. No extra memory is used. The sorting process is performed using only a few extra variables to keep track of values and indices. No extra memory is used. The sorting process is performed using only a few extra variables to keep track of values and indices. No extra memory is used.

It is not a stable sorting algorithm because the process of swapping two elements can change the relative order of elements that have the same value. For example, if an array contains [2, 2, 1], where 1 and 2 are identical elements with different original positions, the sorted array might become [1, 2, 2], which shows that the original relative order is not preserved.

## DRY RUN:

Let's take an array: [7, 4, 5, 4, 9, 3, 2]

After 1st pass: [2, 4, 5, 4, 9, 3, 7]

After 2nd pass: [2, 3, 5, 4, 9, 7, 7]

After 3rd pass: [2, 3, 4, 5, 9, 7, 7]

After 4th pass: [2, 3, 4, 5, 7, 9, 7]

After 5th pass: [2, 3, 4, 5, 7, 9, 7]

After 6th pass: [2, 3, 4, 5, 7, 9, 7]

Similarly for the next iterations:

After 3rd pass: [2, 3, 4, 5, 9, 7, 7] (min\_element = 4)

After 4th pass: [2, 3, 4, 5, 7, 9, 7] (min\_element = 5)

After 5th pass: [2, 3, 4, 5, 7, 9, 7] (min\_element = 7)

After 6th pass: [2, 3, 4, 5, 7, 9, 7] (min\_element = 9)

Final: [2, 3, 4, 5, 7, 9, 7]

## Summing up:

~ In every round, choose the smallest element from the unsorted portion. Place it at its correct position. Repeat until the array is sorted.

~ Selection Sort is NOT suitable for large datasets.

It is useful when:

Memory is very limited.

Swapping elements is expensive.

The dataset size is small.

You want minimal number of swaps.

~ Selection Sort performs at most  $(n-1)$  swaps. Each pass fixes exactly one element in its correct position. This is an advantage over Bubble Sort, which may perform many swaps.

Can Selection Sort Be Optimized?

Not significantly in its basic form. The number of comparisons cannot be reduced. Time complexity remains  $O(n^2)$ .

However, it can be made stable by shifting elements instead of swapping.

~ Time complexity depends mainly on comparisons, not swaps. The number of comparisons is fixed. Even if the array is already sorted, all comparisons still occur.

"Selection Sort minimizes swaps, but not comparisons."

Q. Can we find the k-th smallest element using Selection sorting algorithm?

Yes.

After the first pass, smallest element is fixed.

After k passes, k-th smallest element is fixed.

So we can stop after k passes if needed.

So,  
Insertion Sort is?

So,  
Bubble Sort is?